

1 Key Concepts

1.1 Dynamic Programming

Recall that the core idea of divide-and-conquer is to subdivide a large problem instance into smaller instances of the same problem. For some problems, this recursive subdivision may result in *many re-occurrences of the same subinstances*. Such situations are amenable to the paradigm of dynamic programming.

Dynamic programming is applicable to problems that have the following features:

1. **The principle of optimality**, also known as an “optimal substructure.” This means that *some* (though perhaps not *every*) optimal solution to a larger instance is made up of optimal solution(s) to one or more smaller subinstances.
2. **Overlapping subproblems/subinputs**. This means that the same subinputs occur many times when recursively decomposing the original instance down to the base case(s).

The DP “cookbook.” When formulating any dynamic programming algorithm, there are some common steps you can follow to help structure your approach.

1. First focus on the “**value version**” of the problem. For an optimization problem, temporarily put aside the goal of finding an optimal solution itself, and simply aim to find the value of an optimal solution (e.g., the lowest cost, the largest total value, etc.). In some cases, the original problem is already stated in a value version—e.g., count the number of objects meeting certain constraints—so there is nothing to do in this step.
2. Devise a **recurrence for the value in question**, including base case(s), by understanding how an optimal solution can be made up of solutions to appropriate subinputs. This step usually requires some creativity and insight, both to discover a good set of relevant subinputs, and to see how optimal solutions are related across subinputs.
3. By understanding the dependencies between subinputs as given by the recurrence, **implement the recurrence in (pseudo)code** that fills a table in a bottom-up fashion. Given the recurrence, this step is usually fairly “mechanical” and does not require a great deal of creativity. Answering the following questions may help you structure your algorithm and table:
 - What subinputs can appear in the recurrence? Each subinput typically requires its own cell in the table, so what is the resulting size of the table?
 - To fill in a cell using the recurrence, which other cells need to be filled already (i.e., which other subinputs does the cell’s value depend on)? Based on this, what is a valid order in which to fill the table?
 - Which cell(s) contains the final answer for the original input?
4. If applicable, extend the pseudocode to **solve the original problem using “backtracking”** (instead of just solving the value problem). This involves “following breadcrumbs” (or backpointers) from the final answer to the base case. Given the recurrence and an understanding of why it is correct, this is also usually quite mechanical.

5. Perform a **runtime analysis** of the algorithm. This is also usually fairly mechanical, and follows from analyzing the number of table entries that are filled, and the amount of time it takes to fill each entry (or in some cases, all the entries in total).

2 Discussion Problems

1. Climb Stairs

We define the function $\text{CLIMB-STAIRS}(n)$ as follows: There are n steps to climb. Return the number of ways there are for a person standing at the bottom to reach the top if they can climb either 1, 2 or 3 steps at a time.

Devise a recurrence for computing $\text{CLIMB-STAIRS}(n)$.

Solution: For an input number n , define the function $\text{CS}[n]$ to be the number of ways there are to reach the n th step from the bottom if only 1, 2, or 3 steps can be climbed at a time. For DP problems, it is often helpful to think about base cases. This usually involves thinking about what the algorithm would do when given the simplest or smallest inputs possible. First of all, if there are 0 steps, there is exactly 1 way to get to the top because the top is the bottom and we are already there. If there is just 1 step, there is exactly 1 way to get to the top, which is by climbing up one step from the bottom. If there are 2 steps, we can get to the top by taking 1 step twice or by taking 2 steps once. Next, it is helpful to think of subproblems within the larger problem. When $n \geq 3$, we can only get to the n -th stair in the next move by either stepping up from the $(n-1)$ th step, the $(n-2)$ th step, or the $(n-3)$ th step. This is because we are only allowed to climb either 1, 2, or 3 steps at a time. So the number of ways to get to the n th step would simply be $\text{CS}[n-1] + \text{CS}[n-2] + \text{CS}[n-3]$.

This yields the recurrence

$$\text{CS}[n] = \begin{cases} 1 & \text{if } n \leq 1, \\ 2 & \text{if } n = 2, \\ \text{CS}[n-1] + \text{CS}[n-2] + \text{CS}[n-3] & \text{otherwise.} \end{cases}$$

2. McNuggets

We have a collection M of chicken McNuggets meals; these meals are displayed to you in a menu, represented as an array $M[1, \dots, n]$, with the number of McNuggets per meal. Your goal is to determine, for a given positive integer t , whether it is possible to purchase *exactly* t McNuggets.¹ For example, for $M = [4, 6, 10, 20]$ and $t = 22$, it is possible with $M[2] + M[2] + M[4] = 22$; however, for the same M and $t = 13$, it is not possible.²

¹This problem is also explained in this classic Numberphile video: <https://www.youtube.com/watch?v=vNTSugyS038>

²Fun aside: in the UK, excluding happy meal boxes, there are 3 options for McNuggets: $M = [6, 9, 20]$. The greatest number of McNuggets that *cannot* be exactly purchased is 43 (as explained in the Numberphile video). In the US, the options are $M = [4, 6, 10, 20]$. The greatest number that cannot be purchased is unbounded!

Give a recurrence relation (including base cases), that is suitable for a dynamic programming solution to solve this problem in $O(t \cdot n)$ time. Your solution should include an explanation of why the recurrence is correct.

Solution: We define a function $S(i)$ that is **true** or **false** depending on if it is possible to purchase exactly i McNuggets.

The base cases are:

- If $i = 0$, then $S(i) = \text{true}$. It is possible to purchase 0 McNuggets by purchasing nothing at all.
- If $i < 0$, then $S(i) = \text{false}$. It is impossible to purchase negative McNuggets. We cannot sell McNuggets back (in this problem).

Then for the recursive case, for a given target total of McNuggets i , where $i > 0$, we can make the following inferences:

- If $S(i - M[1])$ is true, then $S(i)$ is true. I.e., if it is possible to purchase $i - M[1]$ McNuggets, then it is possible purchase i McNuggets. This is because I can first purchase $M[1]$ McNuggets, then purchase the rest $i - M[1]$ for a total of i .
- If $S(i - M[2])$ is true, then $S(i)$ is true.
- ...
- If $S(i - M[n])$ is true, then $S(i)$ is true.

Translating this into a recurrence, we have

$$S(i) = \bigvee_{j=1}^n S(i - M[j])$$

A bottom-up implementation of the above algorithm could create a t size table, that tracks the value of $S(i)$ for each $i = 1, \dots, t$. Each step for filling out the table takes n steps, which makes a total $O(tn)$ time.

3. Tilings

Luffy wants to tile a path of length n using k types of blocks with distinct lengths $\ell_1, \dots, \ell_k$ (note that we have an infinite supply of each type of block). He wants to know the total number of ways of tiling such a path. You aim to devise an algorithm for answering this question in $O(nk)$ time.

As a starting point, let $\text{TILINGS}(i)$ be the total number of ways to tile a path of length i .

- (a) Give the recursive formulation for $\text{TILINGS}(i)$ and briefly argue why it is correct. Be sure to include the base case(s).

Solution: Our base case is $n = 0$. There is one way to tile a path of length $n = 0$: by using zero tiles, so $\text{TILINGS}(0) = 1$

Now consider path length $i > 0$ and some arbitrary block length ℓ_j where $1 \leq j \leq k, i$. Assume that ℓ_j is the last tile placed in the path. Then $\text{TILINGS}[i - \ell_j]$ tells us the number of ways to tile the path. We can try this for each of the k blocks that are at most the length of the path.

Our recurrence relation is thus: $\text{TILINGS}(i) = \sum_{j=1}^k \text{TILINGS}(i - \ell_j)$ (only when $i - \ell_j \geq 0$)

- (b) Implement a bottom-up dynamic programming algorithm that solves this problem. Provide pseudocode and analyze the runtime of your algorithm.

Solution: Below is the pseudo-code corresponding to the recurrence.

Input: Path length $n \geq 0$, Block lengths $\{\ell_1, \dots, \ell_k\}$

```

1: function TILINGS( $n, \{\ell_1, \dots, \ell_k\}$ )
2:   initialize array table of length  $n$ 
3:   table[0] = 1
4:   for  $i = 1$  to  $n$  do
5:     table[ $i$ ] = 0
6:     for  $j = 1$  to  $k$  do
7:       if  $i - \ell_j \geq 0$  then
8:         table[ $i$ ] = table[ $i$ ] + table[ $i - \ell_j$ ].
9:   return table[ $n$ ]
```

The running time is $O(nk)$ because of the two nested loops of n and k iterations, respectively. Each iteration takes $O(1)$ time.

4. Longest arithmetic subsequence.

Give a recurrence relation (including base cases) that is suitable for dynamic programming solutions to the following problem. You do not need to prove your correctness.

LONGEST-ARITHMETIC-SUBSEQUENCE(A, d): Given an array of integers A and a difference d , return the length of any longest arithmetic subsequence in A with difference d . That is, return the length of a longest (not necessarily contiguous) subsequence S of A such that $S[i + 1] - S[i] = d$ for each i .

Solution: Let $L[i]$ be a function returning the length of the longest arithmetic subsequence with difference d ending at a particular index i of the array A (where $1 \leq i \leq n$ and the input array is $A[1 \dots n]$). We can look at the i -th element of A and check to see if there are numbers before it that are smaller than it by the exact difference d . By doing this we can reduce the larger problem mentioned before. $L[i]$ should simply return $1 +$

$\max\{L[j_1], L[j_2], \dots, L[j_k]\}$ where j_1 to j_k are the indices of the numbers in A that appear before i and are smaller than $A[i]$ by d . If there are no such j then we know that the longest arithmetic subsequence with difference d ending at i must have length 1 (itself). According to this reasoning, the recurrence relation for $L[i]$ should be:

Let $j_1 < j_2 < \dots < j_k$ be a list of all j such that $0 < j < i$ and $A[i] - A[j] = d$. Then

$$L[i] = \begin{cases} 1 + \max_{m=1}^k (L[j_m]) & \text{if } k \geq 1, \\ 1 & \text{otherwise.} \end{cases}$$

In fact, note that for $1 \leq m < k$, any arithmetic subsequence that end at j_m can be modified to end at j_k by replacing j_m by j_k . This implies $\max_{m=1}^k (L[j_m]) = L[j_k]$, so we can obtain the cleaner recurrence

$$L[i] = \begin{cases} 1 + L[j_k] & \text{if } k \geq 1, \\ 1 & \text{otherwise.} \end{cases}$$

From here, we now have an understanding of a single instance in an array. We must now consider how this can play a role for the larger problem at hand: let $\text{LAS}[A[1 \dots n]]$ be a function returning the length of the longest arithmetic subsequence with difference d in the array A . The length of this sequence can be found by choosing the maximum $L[i]$ (where $1 \leq i \leq n$). Hence, $\text{LAS}[A[1 \dots n]] = \max_{i=1}^n L[i]$.

A linear solution can also be found assuming a perfect hash table.